

Apptainer Container Platform

These directions assume you are already logged into `sporcssubmit.rc.rit.edu` or have accessed a shell from `ondemand.rc.rit.edu` and have entered an **sinteractive** environment.

1. Quick Start:

Get a container definition file from [docker.io](https://docs.docker.com/get-docker/).

```
$ apptainer pull docker://docker.io/ubuntu:jammy
```

Build the container in your home directory.

```
$ apptainer build --fakeroot --sandbox ~/tmp/ubuntu docker://docker.io/ubuntu:jammy
```

Access the container shell as root.

```
$ apptainer shell -f -u --writable ~/tmp/ubuntu
```

To test the container, try installing vim, git, and python3

```
$ apt-get update && apt-get install -y vim git python3
```

2. Load Apptainer Module:

```
$ spack load apptainer
```

3. Downloading and Running Commands Inside the Pulled Containers:

This structure lays the foundation for a comprehensive container build process, allowing for a versatile and tailored containerization experience with Apptainer.

```
$ apptainer pull docker://ubuntu:latest
INFO:      Using cached SIF image
```

```
$ apptainer exec ubuntu_latest.sif cat /etc/os-release
INFO:      fuse2fs not found, will not be able to mount EXT3 filesystems
INFO:      underlay of /etc/localtime required more than 50 (69) bind mounts
PRETTY_NAME="Ubuntu 22.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.4 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy`
```

Creating a local SIF image is not mandatory. You can directly execute commands inside a container pulled from the Docker >registry using a single apptainer exec command. This offers a streamlined approach for quick tasks. For example:

```
$ apptainer exec docker://ubuntu:latest cat /etc/os-release
```

4. Building Containers:

This example provides the basics for getting started with building and running containers using Apptainer on the specified computing resources. For this example, we will create an Apptainer definition file (`.def`), a text file specifying the container's environment. This definition file will be for an Ubuntu container and include installations of specific packages.

Create an Apptainer Definition File for Ubuntu

Create a file named `ubuntu.def` with the following content to create an Ubuntu container and install Python.

```
Bootstrap: docker
From: ubuntu:20.04

%post
    apt-get update && apt-get install -y vim git python3
```

This definition file does the following:

- Uses an Ubuntu 20.04 base image from the docker.
- Installs Vim, Git, and Python3 using `apt-get` in the `%post` section, which runs commands inside the container after the base OS has been installed.

To build the container from the definition file, run:

```
$ apptainer build ubuntu.sif ubuntu.def
```

This command creates a container image named `ubuntu.sif` based on the specifications in `ubuntu.def`.

Running the Container

After building the container, you can run it using the `shell`, `exec`, or `run` commands. For example, to open a shell inside your Ubuntu container, use:

```
$ apptainer shell ubuntu.sif
```

Or if you want to run as root:

```
$ apptainer shell --fakeroot ubuntu.sif
```

5. Custom Containers:

Apptainer leverages a Definition File (`.def` file) to outline a build recipe, delineating the steps to construct a custom container. Identified by its `.def` extension, this file specifies the base operating system for the build, software installations, runtime environment variables, including files from the host system, and container metadata. See the [Apptainer Definition Files documentation](#).

Understanding the Definition File

The Apptainer Definition file is divided into two main parts:

- **Header:** Specifies the core operating system to be built within the container.
- **Sections:** Defined by a `%` character followed by the section name, each serving a unique role in the container's construction.

Header

The header outlines the base OS using the `Bootstrap` keyword, selecting the appropriate bootstrap agent to create the base operating system. For instance, using the `docker` bootstrap agent pulls Docker layers from Docker Hub as the starting point.

Sections

Below are the sections that can be included in a `.def` file:

- **%labels:** Assigns metadata such as the author, maintainer, and URL to the container.

```
%labels
    Author
    Maintainer
    URL
```

- **%files:** Copies files from the building environment into the container during the build process.

```
%files
    ./hello_world.c /opt
```

- **%environment:** Defines environment variables for the container at runtime.

```
%environment
export PATH="$PATH:/opt"
```

- **%post:** Executes commands to customize the build, such as system updates and software installations.

```
%post
echo "Installing required packages..."
apt-get update && apt-get -y upgrade
apt-get install -y gcc
apt-get clean
mkdir -p /opt
gcc -o /opt/hello_world.out /opt/hello_world.c -lpthread
```

- **%test:** Validates the container's functionality after the build is complete.

```
%test
/opt/hello_world.out
```

- **%runscript:** Lists commands executed when the container is launched with `apptainer run`.

```
%runscript
echo "The output of your program is:"
/opt/hello_world.out
```

- **%help:** Generates a help file accessible by using `apptainer run-help`.

```
%help
To run the container:
    apptainer run <container_name.sif>

To shell into the container:
    apptainer shell <container_name.sif>

To execute a custom command or app:
    apptainer exec <container_name.sif> <command or app_name>

To view the container's metadata:
    apptainer inspect <container_name.sif>
```

C Code `hello\_world` for the example above:

```
```bash
// HelloWorld Example Script (`hello_world.c`)

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

// Thread function to print a message
void* printMessage(void* threadId) {
 long tid;
 tid = (long)threadId;
 printf("Thread %ld: Hello, RIT!\n", tid);
 pthread_exit(NULL); // Terminate the thread
}

int main() {
 pthread_t threads[2];
 int threadArgs[2];
 int rc;
 long t;
```

```
// Create and run threads
for(t = 0; t < 2; t++) {
 printf("Main: creating thread %ld\n", t);
 threadArgs[t] = t;
 rc = pthread_create(&threads[t], NULL, printMessage, (void *)t);
 if (rc) {
 printf("ERROR; return code from pthread_create() is %d\n", rc);
 exit(-1);
 }
}

// Wait for all threads to complete
for(t = 0; t < 2; t++) {
 pthread_join(threads[t], NULL);
}

printf("Main: program completed. Exiting.\n");
pthread_exit(NULL); // Terminate the main thread
}
```

6. Using GPUs with Apptainer:

The short answer `--nv`

```
$apptainer shell --nv <container_name.sif>
```

This guide will walk you through creating an Apptainer container designed for running PyTorch applications with GPU support. Following these instructions, you'll set up a Python environment capable of GPU-accelerated computations using PyTorch and Tensorflow.

Sample `.def` File: PyTorch with GPU Support

```
Bootstrap: docker
From: pytorch/pytorch:latest

%files
 ./pytorch_gpu_example.py /opt

%post
 echo "Installing Python and PyTorch with GPU support..."
 apt-get update && apt-get install -y python3-pip
 pip3 install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu111

%runscript
 echo "Running PyTorch GPU example..."
 python3 /opt/pytorch_gpu_example.py
```

PyTorch GPU Example Script ( `pytorch_gpu_example.py` )

Create a Python script named `pytorch_gpu_example.py` with the following content to test GPU support:

```
import torch

def main():
 if torch.cuda.is_available():
 print(f"CUDA is available! Running on GPU: {torch.cuda.get_device_name(0)}")
 else:
 print("CUDA is not available. Running on CPU.")

if __name__ == "__main__":
 main()
```

## Building the Container

Assuming your definition file is named `pytorch_gpu.def` and you have the Python script `pytorch_gpu_example.py` ready, build the container with:

```
$ apptainer build pytorch_gpu.sif pytorch_gpu.def
```

## Running the Container with GPU Support

To run the PyTorch example ensuring GPU access, use the `--nv` flag:

```
$ apptainer run --nv pytorch_gpu.sif
```

This command executes the containerized PyTorch script with access to the host's NVIDIA GPUs, allowing for GPU-accelerated computations.

## Additional Commands for GPU Containers

- **Shell Access with GPU Support:**

```
apptainer shell --nv pytorch_gpu.sif
```

- **Executing a Custom Python Script with GPU Support:**

```
apptainer exec --nv pytorch_gpu.sif python3 /path/to/your_script.py
```

- **Inspecting Container Metadata:**

```
apptainer inspect pytorch_gpu.sif
```

## Sample `.def` File: TensorFlow with GPU Support

```
Bootstrap: docker
From: tensorflow/tensorflow:latest-gpu-jupyter # latest release w/ GPU support and Jupyter

%files
 ./tf_gpu_example.py /opt

%post
 echo "Installing Python and PyTorch with GPU support..."
 apt-get update && apt-get install -y python3-pip

%runscript
 echo "Running TensorFlow GPU example..."
 python3 /opt/tf_gpu_example.py
```

## Tensorflow GPU Example Script ( `tf_gpu_example.py` )

Create a Python script named `tf_gpu_example.py` with the following content to test GPU support:

```
import tensorflow as tf
from tensorflow import keras
import time

class TimeStopping(tf.keras.callbacks.Callback):
 def __init__(self, seconds=None, verbose=0):
 super(TimeStopping, self).__init__()
 self.seconds = seconds
 self.verbose = verbose
 self.start_time = 0

 def on_train_begin(self, logs=None):
 self.start_time = time.time()

 def on_epoch_end(self, epoch, logs=None):
 if time.time() - self.start_time > self.seconds:
```

```

 self.model.stop_training = True
 if self.verbose:
 print(f'Stopping training after {self.seconds} seconds.')

def get_compiled_model():
 # Make a simple 2-layer densely-connected neural network.
 inputs = keras.Input(shape=(784,))
 x = keras.layers.Dense(256, activation="relu")(inputs)
 x = keras.layers.Dense(256, activation="relu")(x)
 outputs = keras.layers.Dense(10)(x)
 model = keras.Model(inputs=inputs, outputs=outputs)
 model.compile(
 optimizer=keras.optimizers.Adam(),
 loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
 metrics=[keras.metrics.SparseCategoricalAccuracy()],
)
 return model

def get_dataset():
 batch_size = 32
 num_val_samples = 10000

 # Return the MNIST dataset in the form of a `tf.data.Dataset`.
 (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

 # Preprocess the data (convert to tensors and normalize)
 x_train = tf.convert_to_tensor(x_train, dtype=tf.float32) / 255.0
 x_test = tf.convert_to_tensor(x_test, dtype=tf.float32) / 255.0
 y_train = tf.convert_to_tensor(y_train, dtype=tf.float32)
 y_test = tf.convert_to_tensor(y_test, dtype=tf.float32)

 # Flatten the input data
 x_train = tf.reshape(x_train, (x_train.shape[0], -1))
 x_test = tf.reshape(x_test, (x_test.shape[0], -1))

 # Reserve num_val_samples samples for validation
 x_val = x_train[-num_val_samples:]
 y_val = y_train[-num_val_samples:]
 x_train = x_train[:-num_val_samples]
 y_train = y_train[:-num_val_samples]

 train_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train)).batch(batch_size)
 val_dataset = tf.data.Dataset.from_tensor_slices((x_val, y_val)).batch(batch_size)
 test_dataset = tf.data.Dataset.from_tensor_slices((x_test, y_test)).batch(batch_size)

 return train_dataset, val_dataset, test_dataset

Specify your total run time in seconds (30 minutes * 60 seconds/minute)
total_run_time_seconds = 30 * 60

Create a MirroredStrategy.
strategy = tf.distribute.MirroredStrategy()
print("Number of devices: {}".format(strategy.num_replicas_in_sync))

Prepare dataset
train_dataset, val_dataset, test_dataset = get_dataset()

Open a strategy scope for model construction and compilation.
with strategy.scope():
 model = get_compiled_model()

Initialize the TimeStopping callback
time_stopping_callback = TimeStopping(seconds=total_run_time_seconds, verbose=1)

Train the model on all available devices with the custom callback.
model.fit(train_dataset, epochs=2, validation_data=val_dataset, callbacks=[time_stopping_callback])

```

```
Test the model on all available devices.
model.evaluate(test_dataset)
```

## Building the Container

Assuming your definition file is named `tf_gpu.def` and you have the Python script `tf_gpu_example.py` ready, build the container with:

```
$ apptainer build tf_gpu.sif tf_gpu.def
```

## Running the Container with GPU Support

To run the PyTorch example ensuring GPU access, use the `--nv` flag:

```
$ apptainer run --nv tf_gpu.sif
```

## Running Apptainer with Slurm

Slurm sbatch example using the TensorFlow example above.

```
#!/bin/bash
#SBATCH --job-name=apptainer_tensorflow_gpu
#SBATCH --account=rc-help
#SBATCH --partition=debug
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=9
#SBATCH --mem=100g
#SBATCH --time=00:30:00
#SBATCH --out=%j-%x.out
#SBATCH --error=%j-%x.err
#SBATCH --mail-user=slack:@$USER
#SBATCH --mail-type=ALL
#SBATCH --gres=gpu:2

spack load --first apptainer # load the first apptainer package listed
apptainer run --nv tf_gpu.sif python -u tf_gpu_example.py
```

## 7. Running Ansys Fluent using Apptainer:

Create File `rocky8.def`

```
BootStrap: docker
From: rockylinux:8
Include: yum

%post
 echo "Doing ROCKY Base now"
 dnf install -y basesystem bash chkconfig coreutils cpio cronie crontabs \
 dhclient dnf e2fsprogs ethtool filesystem findutils fipscheck \
 fipscheck-lib gawk grep gzip initscripts ipmitool iproute iputils \
 less libndp net-tools NetworkManager NetworkManager-libnm nfs-utils \
 openssh-clients openssh-server pam pciutils polkit-libs psmisc \
 rocky-release rsync rsyslog sed setup shadow-utils strace sudo tar \
 tzdata util-linux vim-minimal wget which words zlib \
 --allowerasing
 echo "Doing EPEL now"
 dnf install -y https://mirrors.rit.edu/epel/epel-release-latest-8.noarch.rpm
 echo "Doing Workstation now"
 dnf groupinstall -y Workstation --nobest
 dnf install -y libnsl
```



Finally, make the script executable:

```
chmod +x run_rserver.sh
```

### Running RStudio with OnDemand Desktop

1. Execute the script:

```
./run_rserver.sh
```

2. Open a browser and goto localhost:8787

## Comments